



《AI大模型Ragent项目》——多模型路由与智能选择

来自： 拿个offer-开源&项目实战



2026年04月09日 21:49

上一篇我们从宏观视角看了 infra-ai 模块的整体架构，知道了一次模型调用会经过 `ModelSelector` → `ModelRoutingExecutor` → `ModelHealthStore` → `ChatClient` 这条链路。其中 `ModelSelector` 是链路的起点——它决定了候选列表的顺序，而顺序直接决定了优先调谁、失败了切换到谁。

上一篇用一段话概述了它的工作流程：读取配置 → 过滤禁用 → 按优先级排序 → 提升首选模型 → 排除熔断模型 → 返回有序列表。但没有展示代码，那些步骤里的细节都是黑盒。

这一篇打开 `ModelSelector` 这个黑盒，看看它内部到底是怎么从一份 YAML 配置中选出一个有序的、可用的候选列表的。同时也会讲清楚 `ModelTarget` 的构建过程和 `ModelUrlResolver` 的 URL 解析逻辑——它们和 `ModelSelector` 一起，构成了从配置到实际 HTTP 调用之间的完整桥梁。

ModelSelector 的选择算法

`ModelSelector` 是一个 Spring 组件，依赖注入两个对象：`AIModelProperties`（配置）和 `ModelHealthStore`（熔断器）。

```
@Slf4j
@Component
@RequiredArgsConstructor
public class ModelSelector {

    private final AIModelProperties properties;
    private final ModelHealthStore healthStore;
    // ...
}
```

它对外暴露的方法不多，但每一个背后都有一套完整的选择算法。

1. 入口方法

`ModelSelector` 提供三个入口方法，分别对应三种能力：

```
public List<ModelTarget> selectChatCandidates(boolean deepThinking) {
    AIModelProperties.ModelGroup group = properties.getChat();
    if (group == null) {
        return List.of();
    }

    String firstChoiceModelId = resolveFirstChoiceModel(group, deepThinking);
    return selectCandidates(group, firstChoiceModelId, deepThinking);
}

public List<ModelTarget> selectEmbeddingCandidates() {
    return selectCandidates(properties.getEmbedding());
}

public List<ModelTarget> selectRerankCandidates() {
    return selectCandidates(properties.getRerank());
}
```

Chat 和其他两种能力有一个明显的差异：Chat 多了一个 `deepThinking` 参数。这个参数决定了是走普通模式还是深度思考模式，两种模式选出来的候选列表可能完全不同——不只是顺序不同，连包含哪些候选都不一样。

Embedding 和 Rerank 的选择逻辑是 Chat 的简化版。它们调的是一个重载方法，内部不做深度思考过滤，其他步骤和 Chat 一致：

```
private List<ModelTarget> selectCandidates(AIModelProperties.ModelGroup group) {
    if (group == null) {
```

```
        return List.of();
    }
    return selectCandidates(group, group.getDefaultModel(), false);
}
```

2. 确定首选模型

在正式排序之前，ModelSelector 先确定谁应该排第一位。这个逻辑由 resolveFirstChoiceModel 方法完成：

```
private String resolveFirstChoiceModel(AIModelProperties.ModelGroup group, boolean deepThinking) {
    if (deepThinking) {
        String deepModel = group.getDeepThinkingModel();
        if (StrUtil.isNotBlank(deepModel)) {
            return deepModel;
        }
    }
    return group.getDefaultModel();
}
```

逻辑很直白：如果是深度思考模式且配了 deep-thinking-model，用它；否则用 default-model。
对应到配置：

```
chat:
  default-model: qwen3-max          # 普通模式的首选
  deep-thinking-model: qwen3-max    # 深度思考模式的首选
```

这两个字段的值可以相同也可以不同。如果你希望深度思考走一个专门的推理模型（比如 OpenAI o1、DeepSeek-R1），就把 deep-thinking-model 配成那个模型的 id。如果没配 deep-thinking-model，深度思考模式会回退到 default-model。

注意：resolveFirstChoiceModel 返回的是一个模型 id 字符串，不是模型对象。这个 id 会在后续的首选模型提升步骤中用到。

3. 过滤与排序

确定了首选模型后，进入核心的过滤排序逻辑。整个 selectCandidates 方法分两步：先准备有序候选列表，再构建可用目标。

```
private List<ModelTarget> selectCandidates(
    AIModelProperties.ModelGroup group,
    String firstChoiceModelId,
    boolean deepThinking) {
    if (group == null || group.getCandidates() == null) {
        return List.of();
    }

    List<AIModelProperties.ModelCandidate> orderedCandidates =
        filterAndSortCandidates(group.getCandidates(), firstChoiceModelId, deepThinking);

    return buildAvailableTargets(orderedCandidates);
}
```

filterAndSortCandidates 是整个选择算法中信息密度最高的方法：

```
private List<AIModelProperties.ModelCandidate> filterAndSortCandidates(
    List<AIModelProperties.ModelCandidate> candidates,
    String firstChoiceModelId,
    boolean deepThinking) {

    List<AIModelProperties.ModelCandidate> enabled = candidates.stream()
        .filter(c -> c != null && !Boolean.FALSE.equals(c.getEnabled()))
```

```
.filter(c -> !deepThinking || Boolean.TRUE.equals(c.getSupportsThinking()))
.sorted(Comparator
    .comparing((AIModelProperties.ModelCandidate c) ->
        !Objects.equals(resolveId(c), firstChoiceModelId))
    .thenComparing(AIModelProperties.ModelCandidate::getPriority,
        Comparator.nullsLast(Integer::compareTo))
    .thenComparing(AIModelProperties.ModelCandidate::getId,
        Comparator.nullsLast(String::compareTo)))
.collect(Collectors.toList());

if (deepThinking && enabled.isEmpty()) {
    log.warn("深度思考模式没有可用候选模型");
}

return enabled;
}
```

这段代码在一条 Stream 链里完成了过滤、排序、首选提升三件事，拆开来看：

3.1 过滤禁用候选

```
.filter(c -> c != null && !Boolean.FALSE.equals(c.getEnabled()))
```

enabled 字段默认是 true。只有显式配了 enabled: false 的候选才会被过滤掉。这意味着你可以在不删除配置的情况下临时禁用某个模型——比如某个供应商在做维护，你把它的候选标记为 enabled: false，等恢复了再改回来。

3.2 深度思考模式的额外过滤

```
.filter(c -> !deepThinking || Boolean.TRUE.equals(c.getSupportsThinking()))
```

这行的逻辑是：如果不是深度思考模式（deepThinking 为 false），所有候选都放行；如果是深度思考模式，只放行 supportsThinking = true 的候选。

用自然语言翻译就是：普通模式不做额外过滤；深度思考模式只保留支持深度思考的模型。

3.3 排序规则（含首选模型提升）

```
.sorted(Comparator
    .comparing((AIModelProperties.ModelCandidate c) ->
        !Objects.equals(resolveId(c), firstChoiceModelId))
    .thenComparing(AIModelProperties.ModelCandidate::getPriority,
        Comparator.nullsLast(Integer::compareTo))
    .thenComparing(AIModelProperties.ModelCandidate::getId,
        Comparator.nullsLast(String::compareTo)))
```

排序规则是三级：

1. **主排序：首选模型优先。** !Objects.equals(resolveId(c), firstChoiceModelId) 对首选模型返回 false，对其他候选返回 true。因为 false < true，首选模型永远排在最前面。这里用 resolveId(c) 而不是 c.getId()，是因为 resolveId 会在候选没有显式配 id 时自动合成 "provider::model" 格式的 id，保证比较逻辑和 buildModelTarget 里用的 id 一致。
2. **次排序：**按 priority 升序，数字越小越优先。priority 为 null 的排最后。
3. **末排序：**priority 相同时按 id 字母序排列。id 为 null 的排最后。

把首选模型提升和 priority 排序合并到同一个 Comparator 链里，一次 sorted 就完成了所有排序逻辑，不需要对列表做额外的 remove + add(0, ...) 操作。

拿实际配置举例：default-model 配的是 qwen3-max（priority=3），排序时它通过主排序键的 false 值直接排到第一位，不受 priority 影响。其余候选按 priority 升序排列：glm-4.7 (0) → qwen-plus (1) → qwen3-local (2)。

为什么要加 id 作为末排序？为了保证排序结果的确定性。如果两个候选的 priority 都是 1，每次排序的相对顺序可能不一样（Java 的 sort 不保证稳定性在所有实现中都一致）。加上 id 作为 tie-breaking，排序结果就是确定的——不管跑多少次，同样的配置永远产出同样的顺序。

3.4 为什么不直接把 defaultModel 的 priority 设为最小

你可能会想：既然 qwen3-max 要排第一，直接把它 priority 改成 0 不就行了？为什么要在排序器里单独处理首选模型？

因为 priority 和 defaultModel 的语义不同。

priority 是配置在候选上的静态属性，表示这个模型在整个系统中的一般优先级。比如你有四个 Chat 模型，它们的 priority 反映的是你对它们稳定性、成本、质量的综合排名——0 号最好，100 号垫底。这个排名是长期不变的。

defaultModel 是模型组级别的选择，表示当前默认使用哪个模型。它可能经常变——这周用百炼的 qwen3-max，下周切到硅基流动的 glm-4.7 试试效果。改 default-model 只需要换一个 id，不影响任何模型的 priority。

如果把两者合并到 priority 里，切换默认模型就得调整多个模型的 priority 值，容易出错。分开处理，各管各的事，配置的意图也更清晰。

4. 构建 ModelTarget

候选列表排好序之后，最后一步是把每个 ModelCandidate（配置对象）转换为 ModelTarget（运行时目标对象），同时过滤掉不可用的候选。

```
private List<ModelTarget> buildAvailableTargets(
    List<AIModelProperties.ModelCandidate> candidates) {
    Map<String, AIModelProperties.ProviderConfig> providers = properties.getProviders();

    return candidates.stream()
        .map(candidate -> buildModelTarget(candidate, providers))
        .filter(Objects::nonNull)
        .collect(Collectors.toList());
}
```

buildModelTarget 是构建单个目标的方法：

```
private ModelTarget buildModelTarget(
    AIModelProperties.ModelCandidate candidate,
    Map<String, AIModelProperties.ProviderConfig> providers) {

    String modelId = resolveId(candidate);

    // 检查熔断状态
    if (healthStore.isUnavailable(modelId)) {
        return null;
    }

    // 验证 provider 配置
    AIModelProperties.ProviderConfig provider = providers.get(candidate.getProvider());
    if (provider == null && !ModelProvider.NOOP.matches(candidate.getProvider())) {
        log.warn("Provider配置缺失: provider={}, modelId={}",
            candidate.getProvider(), modelId);
        return null;
    }

    return new ModelTarget(modelId, candidate, provider);
}
```

这个方法里有三个关键逻辑。

第一，ID 解析。resolveId 方法决定每个候选的唯一标识：

```
private String resolveId(AIModelProperties.ModelCandidate candidate) {
    if (StrUtil.isNotBlank(candidate.getId())) {
        return candidate.getId();
    }
    return String.format("%s:%s",
        Objects.toString(candidate.getProvider(), "unknown"),
        Objects.toString(candidate.getModel(), "unknown"));
}
```

规则很简单：如果配置了 `id` 字段，直接用它；如果没配，自动合成 `"provider::model"` 格式。比如一个没配 `id` 的候选，`provider` 是 `bailian`，`model` 是 `qwen-plus`，合成的 `id` 就是 `"bailian::qwen-plus"`。

自动合成的 `id` 可读性差，而且和模型名耦合——换个模型名 `id` 就变了，熔断器里的健康记录也就失效了。所以建议始终在配置中显式指定 `id`。

第二，熔断预过滤。 `healthStore.isUnavailable(modelId)` 检查这个模型是否正处于熔断状态。如果是，直接返回 `null`，这个候选不会出现在最终的目标列表中。这是熔断检查的第一层，后面还会讲为什么需要两层。

第三，供应商配置查找。 从 `providers Map` 中按候选的 `provider` 字段查找对应的 `ProviderConfig`。如果找不到，分两种情况处理。

noop 供应商的特殊处理： `noop` 是一个特殊的供应商，对应 `NoopRerankClient`——一个什么都不做的空实现。它不发 HTTP 请求，不需要 URL，不需要 API Key，所以它的 `ProviderConfig` 可以为 `null`。

看这行判断：

```
if (provider == null && !ModelProvider.NOOP.matches(candidate.getProvider())) {
    log.warn("Provider配置缺失: provider={}, modelId={}", ...);
    return null;
}
```

翻译过来就是：如果供应商配置为空，并且这个供应商不是 `noop`，才报警告并跳过。如果是 `noop`，即使 `provider` 是 `null` 也放行，正常构建 `ModelTarget`。

`noop` 的典型用途是 `Rerank` 的兜底。看配置：

```
rerank:
  candidates:
    - id: qwen3-rerank
      provider: bailian
      priority: 1
    - id: rerank-noop
      provider: noop
      model: noop
      priority: 100
```

如果百炼的 `Reranker` 挂了，故障转移到 `rerank-noop`，它直接按原始顺序截断返回 `topN` 结果——不做重排序，但也不报错。总比整个系统挂掉强。

ModelTarget：运行时调用目标

1. record 定义

`ModelTarget` 的定义非常简洁：

```
public record ModelTarget(
    String id,
    AIModelProperties.ModelCandidate candidate,
    AIModelProperties.ProviderConfig provider
) {}
```

三个字段：

`id`：模型的唯一标识，来自 `resolveId` 的解析结果

`candidate`：模型候选配置对象，包含模型名、优先级、维度等信息

`provider`：供应商配置对象，包含 URL、API Key、端点路径等信息（`noop` 供应商时为 `null`）

用 `record` 而不是普通 POJO 的原因：`ModelTarget` 是一个纯数据载体，在 `ModelSelector`、`ModelRoutingExecutor`、`ChatClient` 等组件之间传递。`record` 天然不可变，自动生成 `equals` / `hashCode` / `toString`，既安全又省代码。

当然了，其实看起来 `record` 和普通 POJO 没有非常明显必须二选一的差距，实际项目中混合使用就好。

2. 从配置到 ModelTarget 的映射

拿配置中的 `qwen-plus` 候选为例，看看 `ModelTarget` 的三个字段分别从哪里来：

ModelTarget 字段	值	来源
id	"qwen-plus"	候选配置的 id 字段，由 resolveId 解析
candidate	ModelCandidate{id="qwen-plus", provider="bailian", model="qwen-plus-latest", priority=1, ...}	候选配置对象本身
provider	ProviderConfig{url="https://dashscope.aliyuncs.com", apiKey="sk-xxx", endpoints={chat: "/compatible-mode/v1/chat/completions", ...}}	从 providers Map 中按 "bailian" 查找

后续的供应商客户端拿到 ModelTarget 后，就能从中获取所有需要的信息：用 candidate.getModel() 拿模型名，用 provider.getUrl() 拿基础 URL，用 provider.getApiKey() 拿认证密钥。

ModelUrlResolver：URL 解析

ModelTarget 封装了候选配置和供应商配置，但最终发 HTTP 请求需要一个完整的 URL。ModelUrlResolver 就是负责把这些配置拼成 URL 的工具类。

1. 解析优先级

```
public static String resolveUrl(
    AIModelProperties.ProviderConfig provider,
    AIModelProperties.ModelCandidate candidate,
    ModelCapability capability) {

    // 优先级一：候选级 URL 覆盖
    if (candidate != null && candidate.getUrl() != null && !candidate.getUrl().isBlank()) {
        return candidate.getUrl();
    }

    // 优先级二：供应商级 URL + 端点路径拼接
    if (provider == null || provider.getUrl() == null || provider.getUrl().isBlank()) {
        throw new IllegalStateException("Provider baseUrl is missing");
    }

    Map<String, String> endpoints = provider.getEndpoints();
    String key = capability.name().toLowerCase();
    String path = endpoints == null ? null : endpoints.get(key);
    if (path == null || path.isBlank()) {
        throw new IllegalStateException("Provider endpoint is missing: " + key);
    }

    return joinUrl(provider.getUrl(), path);
}
```

两级优先级：

- 1. 如果候选配置了自己的 url 字段，直接返回，不走拼接。
- 2. 否则，用供应商的 url + endpoints 中对应能力的路径拼接成完整 URL。

端点路径的查找 key 是能力枚举的小写名：CHAT → "chat"，EMBEDDING → "embedding"，RERANK → "rerank"。
用几个具体例子说明：

候选	候选 url	供应商 url	端点路径	解析结果
qwen-plus	未配置	https://dashscope.aliyuncs.com	chat → /compatible-mode/v1/chat/completions	https://dashscope.aliyuncs.com/compatible-mode/v1/chat/completions

glm-4.7	未配置	https://api.siliconflow.cn	chat → /v1/chat/completions	https://api.siliconflow.cn/v1/chat/completions
qwen3-local	未配置	http://localhost:11434	chat → /v1/chat/completions	http://localhost:11434/v1/chat/completions
私有部署	http://10.0.1.100:8000/v1/chat/completions	(忽略)	(忽略)	http://10.0.1.100:8000/v1/chat/completions

2. 斜杠拼接的边界处理

joinUrl 方法处理 baseUrl 和 path 之间的斜杠问题：

```
private static String joinUrl(String baseUrl, String path) {
    if (baseUrl.endsWith("/") && path.startsWith("/")) {
        return baseUrl + path.substring(1);
    }
    if (!baseUrl.endsWith("/") && !path.startsWith("/")) {
        return baseUrl + "/" + path;
    }
    return baseUrl + path;
}
```

四种情况：

baseUrl 结尾	path 开头	处理方式	示例
有 /	有 /	去掉 path 开头的 /	https://api.com/ + /v1/chat → https://api.com/v1/chat
无 /	无 /	中间插入 /	https://api.com + v1/chat → https://api.com/v1/chat
有 /	无 /	直接拼接	https://api.com/ + v1/chat → https://api.com/v1/chat
无 /	有 /	直接拼接	https://api.com + /v1/chat → https://api.com/v1/chat

这是一个小细节，但很容易出 bug。配置文件里有人喜欢在 URL 末尾加斜杠，有人不加，端点路径有人喜欢以斜杠开头，有人不加。joinUrl 兼容了所有组合，不管怎么配都能拼出正确的 URL。

3. 候选级 URL 覆盖的使用场景

什么时候需要用候选级 URL 覆盖？
最常见的场景是私有部署。比如你在内网部署了一个 vLLM 推理服务，地址是 http://10.0.1.100:8000/v1/chat/completions，它和任何公共供应商都没关系。这时候在候选上配一个 url 字段：

```
candidates:
- id: vllm-local
  provider: ollama          # 协议兼容 OpenAI，复用 Ollama 客户端
  model: my-fine-tuned-model
  url: http://10.0.1.100:8000/v1/chat/completions
  priority: 50
```

ModelUrlResolver 看到候选有 url，直接返回，不去拼接供应商的 URL 和端点。
另一个场景是同一个供应商的不同模型走不同的网关地址。有些供应商对大模型和小模型用不同的 API 端点，或者你有一个 API 代理层对不同模型做了分流。这时候可以给个别候选配上专属的 url，其他候选继续走供应商的默认拼接。

熔断检查的两层机制

前面提到，`buildModelTarget` 里调了 `healthStore.isUnavailable(modelId)` 来过滤熔断模型。但如果你看过上一篇的时序图，会发现 `ModelRoutingExecutor.executeWithFallback()` 里还有一次 `healthStore.allowCall(target.id())` 检查。同一个熔断器，为什么要检查两次？

1. 第一层：选择阶段预过滤（isUnavailable）

`isUnavailable` 在 `ModelSelector.buildModelTarget()` 中被调用，发生在构建候选列表的阶段：

```
// ModelSelector.buildModelTarget()
if (healthStore.isUnavailable(modelId)) {
    return null; // 不进入候选列表
}
```

它是一个只读检查，不修改任何状态。看看它的实现：

```
public boolean isUnavailable(String id) {
    ModelHealth health = healthById.get(id);
    if (health == null) {
        return false;
    }
    if (health.state == State.OPEN && health.openUntil > System.currentTimeMillis()) {
        return true;
    }
    return health.state == State.HALF_OPEN && health.halfOpenInFlight;
}
```

两种情况返回 `true`（不可用）：

模型处于 `OPEN` 状态且冷却时间未到

模型处于 `HALF_OPEN` 状态且已有一个探测请求在调用中

这层检查的目的是性能优化：在构建候选列表时就排除掉明确不可用的模型，让它们不出现在 `executeWithFallback` 的遍历列表中。这样可以减少无效的客户端查找和调用准备开销。

2. 第二层：调用阶段最终判断（allowCall）

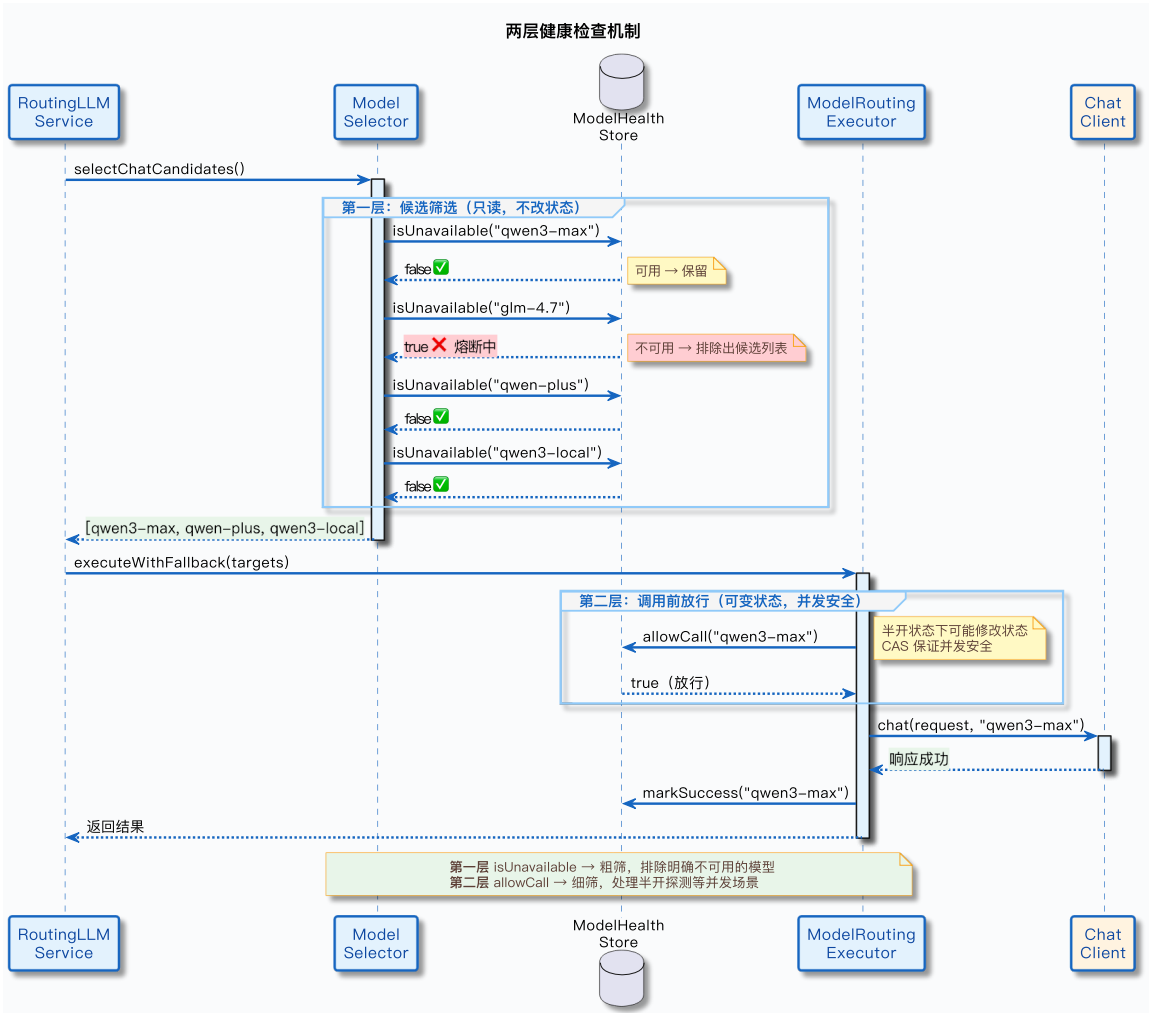
`allowCall` 在 `ModelRoutingExecutor.executeWithFallback()` 中被调用，发生在实际发起 HTTP 调用之前：

```
// ModelRoutingExecutor.executeWithFallback()
if (!healthStore.allowCall(target.id())) {
    continue; // 跳过这个候选
}
```

和 `isUnavailable` 不同，`allowCall` 不只是读状态，它还会修改状态。比如当模型处于 `OPEN` 状态且冷却时间已到，`allowCall` 会将状态从 `OPEN` 切换到 `HALF_OPEN` 并设置 `halfOpenInFlight = true`，表示允许一个探测请求通过。这个状态转换是通过 `ConcurrentHashMap.compute()` 原子完成的，保证并发安全。

3. 为什么需要两层

用一张时序图来看两层检查在调用链路中的位置：



总结两层检查的分工：

	第一层 isUnavailable	第二层 allowCall
调用位置	ModelSelector.buildModelTarget()	ModelRoutingExecutor.executeWithFallback()
目的	性能优化：减少无效遍历	正确性保证：并发安全的最终判断
是否修改状态	否（只读）	是（可能触发 OPEN → HALF_OPEN 转换）
缺了会怎样	每次都要为已熔断的模型做客户端查找等准备工作	并发场景下可能调到刚熔断的模型

如果只有第一层，从 ModelSelector 返回候选列表到 executeWithFallback 实际调用之间可能有时间差，模型的健康状态可能已经变化（比如另一个线程刚标记了它熔断）。第二层的 allowCall 用 ConcurrentHashMap.compute() 做原子检查，保证了最终判断的正确性。如果只有第二层，每次调用都要为所有已熔断的模型做客户端查找、ModelTarget 构建等准备工作，然后在 allowCall 那里才发现不能调，白忙一场。第一层提前过滤，避免了这些无效开销。

ModelHealthStore 的三态状态机实现细节（ConcurrentHashMap.compute() 如何保证并发安全、markSuccess / markFailure 的状态转换逻辑）留给下一篇专门讲，这里只需要知道两层检查各自的职责。

完整走查：从配置到候选列表

说了这么多方法和逻辑，用具体的配置把整个流程串一遍。基于前面的 YAML 配置，分三个场景走查。

1. 场景一：普通 Chat 调用

业务层调用 llmService.chat("AirPods Pro 2 的保修期是多久? ")，请求中没有指定深度思考，deepThinking 为 false。

步骤 1：确定首选模型

resolveFirstChoiceModel(group, false): deepThinking 为 false，跳过深度思考分支，返回 group.getDefaultModel() = "qwen3-max"。

步骤 2：过滤与排序

原始候选列表（4 个）：

id	provider	priority	supportsThinking	enabled
glm-4.7	siliconflow	0	true	true
qwen-plus	bailian	1	false	true
qwen3-local	ollama	2	false	true
qwen3-max	bailian	3	true	true

过滤 enabled = false: 全部启用, 无变化, 仍然 4 个。
深度思考过滤: deepThinking 为 false, 不做额外过滤, 仍然 4 个。
三级排序: qwen3-max 匹配首选模型, 主排序键为 false, 排在最前; 其余候选按 priority 升序排列:

顺序	id	priority	说明
1	qwen3-max	3	← 首选模型, 主排序键 false
2	glm-4.7	0	
3	qwen-plus	1	
4	qwen3-local	2	

虽然 qwen3-max 的 priority 是 3 (最大), 但因为它是 default-model, 通过主排序键直接排到了第一位。

步骤 3: 构建 ModelTarget

假设所有模型健康 (无熔断), 所有供应商配置存在。四个候选都成功构建为 ModelTarget, 最终返回的候选列表顺序就是: qwen3-max → glm-4.7 → qwen-plus → qwen3-local。
executeWithFallback 拿到这个列表后, 会按顺序逐个尝试: 先调 qwen3-max, 如果失败了切到 glm-4.7, 再失败切到 qwen-plus, 最后兜底到本地的 qwen3-local。

2. 场景二：深度思考 Chat 调用

业务层调用时指定了深度思考模式, deepThinking = true。

步骤 1: 确定首选模型

resolveFirstChoiceModel(group, true): deepThinking 为 true, 检查 group.getDeepThinkingModel() = "qwen3-max", 非空, 返回 "qwen3-max"。

步骤 2: 过滤与排序

原始候选列表同上 (4 个)。
过滤 enabled = false: 无变化。
深度思考过滤 (supportsThinking = true 才保留):

id	supportsThinking	是否保留
glm-4.7	true	保留
qwen-plus	false	过滤掉
qwen3-local	false	过滤掉
qwen3-max	true	保留

过滤后只剩 2 个候选。三级排序: qwen3-max 匹配首选模型, 排在最前:

顺序	id	priority	说明
1	qwen3-max	3	← 首选模型, 主排序键 false
2	glm-4.7	0	

步骤 3: 构建 ModelTarget

假设无熔断, 两个候选都成功构建。最终返回: qwen3-max → glm-4.7。
对比两个场景的结果:

	普通模式	深度思考模式
候选数量	4 个	2 个

候选列表	qwen3-max → glm-4.7 → qwen-plus → qwen3-local	qwen3-max → glm-4.7
兜底方案	有本地 Ollama 兜底	只有 2 个支持深度思考的模型

深度思考模式下候选更少，因为不是所有模型都支持深度思考。如果你需要更多兜底选项，可以在配置中给更多模型标记 `supports-thinking: true`。

3. 场景三：有模型被熔断时

回到普通模式，但这次假设 `qwen3-max` 被熔断了（连续失败 2 次，`isUnavailable` 返回 `true`）。
步骤 1~2 和场景一完全一样，排序后的候选列表仍然是：`qwen3-max → glm-4.7 → qwen-plus → qwen3-local`。
到步骤 3，`buildAvailableTargets` 遍历每个候选调用 `buildModelTarget`：

qwen3-max: `healthStore.isUnavailable("qwen3-max") → true`，返回 `null`，被过滤掉
glm-4.7: 健康，正常构建
qwen-plus: 健康，正常构建
qwen3-local: 健康，正常构建

最终返回：`glm-4.7 → qwen-plus → qwen3-local`。默认模型被跳过了，直接从下一个候选开始。
这个例子说明了配置驱动 + 熔断保护的协作效果：`priority` 和 `defaultModel` 定义了正常情况下的优先级顺序，`ModelHealthStore` 在运行时动态排除故障模型。整个过程对业务层完全透明——业务层调用 `llmService.chat(...)` 拿到的永远是一个可用模型的响应，它不需要知道背后发生了什么切换。
等 30 秒冷却时间过后，`qwen3-max` 进入 `HALF_OPEN` 状态，允许一个探测请求通过。如果探测成功，它又回到 `CLOSED` 状态，下一次 `selectChatCandidates` 时就会重新出现在候选列表中。

小结与下一步

回顾一下这篇文章的核心要点：

- `ModelSelector` 的选择算法分四步：确定首选 → 过滤禁用和不支持的候选 → 按首选优先 + `priority` 三级排序 → 熔断预过滤。每一步都有明确的职责
- `priority` 控制候选的一般优先级顺序，`defaultModel` / `deepThinkingModel` 控制首选模型提升。两者语义不同，在排序器中分层处理，互不干扰
- 深度思考模式通过 `supportsThinking` 过滤和 `deepThinkingModel` 提升两步操作，生成和普通模式完全不同的候选列表
- `ModelTarget` 是从配置到运行时的桥梁，把候选配置和供应商配置打包传递给供应商客户端
- `ModelUrlResolver` 支持两级 URL 解析：候选级覆盖优先于供应商级拼接，兼容各种斜杠组合
- 熔断检查分两层：选择阶段的 `isUnavailable`（只读预过滤，性能优化）和调用阶段的 `allowCall`（原子状态转换，正确性保证）

`ModelSelector` 解决了选谁的问题，但选好了之后怎么调、调失败了怎么切换、什么时候该熔断——这些都是下一篇的内容。下一篇我们深入 **熔断器与故障转移**——拆解 `ModelHealthStore` 的三态状态机实现，看它如何用 `ConcurrentHashMap.compute()` 保证并发安全的状态转换；再看 `ModelRoutingExecutor.executeWithFallback()` 的故障转移执行逻辑，以及流式调用中最复杂的首包探测机制。